

Fuzzycomplete

Fuzzy SQL completion with a catalog-aware matcher

Aa

Install & load

```
INSTALL fuzzycomplete FROM community;  
LOAD fuzzycomplete;
```

01 At a glance

Suggest SQL keywords and catalog names from a partial query using a fuzzy matcher. Fixed v1.5 builds activate even when the DuckDB CLI has already loaded stock autocomplete; inspect suggestions through the table function.

Partial SQL → Catalog + matcher → Suggestions

02 Completion surface

`sql_auto_complete(query)`

Table function: suggest completions for partial SQL text.

`suggestion`

Candidate text returned by the completion provider.

`suggestion_start`

Character offset at which the candidate should replace text.

`Tab` in the CLI

Requests completion through the active provider.

`duckdb_extensions()`

Check whether autocomplete was already loaded.

`duckdb_functions()`

Inspect the registered completion function and its parameters.

03 Keep in mind

Activation depends on the build

Fixed v1.5 builds replace preloaded stock autocomplete. Older 9746833 skips replacement when autocomplete is loaded. Check behavior with SLCT: the fuzzy matcher suggests SELECT. Loaded=true alone does not identify the active provider.

04 Inspect the loaded provider

```
SELECT extension_name, loaded  
FROM duckdb_extensions()  
WHERE extension_name IN  
('autocomplete', 'fuzzycomplete');
```

Reports loaded extensions; use the suggestion check below to verify fuzzy behavior.

05 Inspect SQL suggestions

```
SELECT trim(suggestion) AS candidate  
FROM sql_auto_complete('SLCT');  
-- SELECT
```

A subsequence query checks the actual fuzzy provider on the fixed build.

06 Try a catalog abbreviation

MANUAL SETUP

```
CREATE TEMP TABLE getNotificationStatus(  
  id INTEGER  
);  
-- In a compatible interactive CLI, type:  
-- SELECT * FROM gnst  
-- Then press Tab before submitting.
```

The client must call DuckDB's active completion provider for Tab.

07 In practice

Matcher and client are separate

Fuzzy matching rewards ordered character matches and word boundaries. SQL calls inspect suggestions; they do not add an editor to Python, JDBC, or other clients. Interactive Tab behavior depends on the client and active provider.

[Full docs & examples ↗](#) [Source ↗](#)